So; we define the tuple; $\{S, A, R, \gamma\}$ in addition to the ~~~~ policy shape

1st of all; γ was set to be = 1, the reason for that is that having it as $\gamma = 0.99$ will cause the return ~~to~~ $G_t = \gamma r_t + \gamma^2_{t+1} + \gamma^2 r_{t+2} \cdots$ to ~~be~~ say I should value points at the beginning of the simulations more than those at the end.

and in theory, for episodic simulations, $\gamma = 1$ is always acceptable.

either way, γ was tested for different values before we settled for 1 as it was converging.

for the state, what is taken to be the state ~~~~ are all the particles coordinates + the target pt

thus ~~~~ $S := \{x_i, t_i, u_i, x^*, t^*\}$

thus, based on the number of ~~~~ particles ~~~~ being N; we'll have the state size be $\{3N+2\}$, however the +2 i.e. the (x^*, t^*) are still debatable because when we feed the agent 2 numbers like (0.05, 0.1) how is it supposed to relate it to the dt size we are taking & thus that this is

is the target point. anyway, we'll test that later.
Moving to the reward. for now, the reward is terminal. sparse reward

=> when we started building the toy problem, it was set to be:

if target point was reached => r = -(abs(error)) - λK

where the error = Upred - Uactual. & K is the number of steps we took to reach the target. & λ = 0.001 to balance between the order of magnitudes.

however later after testing, we noticed that λ was not enough

* if target = not reached. ; i.e; K > Kmax

r = -1 ; terminate run

*later it was noticed that 2 bugs we in the code.

1st is, for this toy problem error are in the order of e^{-6}, while. the $-\lambda K_{max}$ was in the order of of e^{0} So the accuracy wasn't giving any signal

So we added another constant to the accuracy beta(b) where $\boxed{b = 1000}$ also, for K > Kmax case

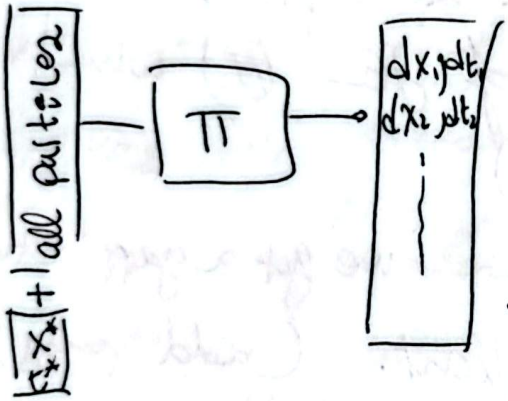
reward = $-1 - \lambda K_{max}$ to be able to give it more signal ever in the non-reach case.

e

for the action, 1st, because we wanted it to be only for testing $A = \{0:\{0,0\}, 1:\{0,1\}, 2:\{1,0\}, 3:\{1,1\}\}$ however, this was later changed due to the Policy shape. which is discussed in the following section:

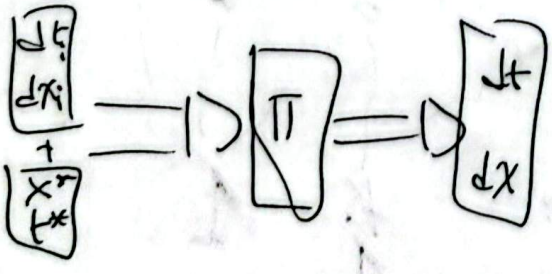
Policy:

1st; policy was chosen to be a shared policy



So, we input all the particles positions + values, in addition to the target values + it outputs all the dts & dxs of all points. as discrete test.

but this proved to be not working because it needs to choose from a combination of dx(3), dt(3) so 6 X (number of particles) branch & state value => # So we switched to another technique where we only input one particle at a time



only, which showed some promising results

but it defeats the purpose of having N particles in the

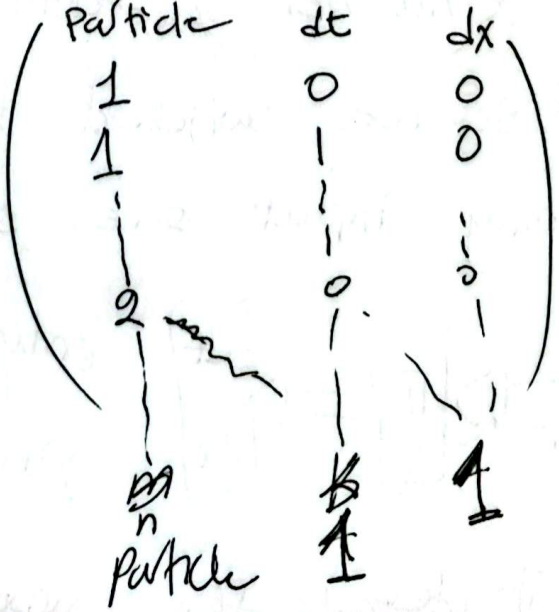
1st place. because this can be thought of as 1 particle moving and at each time the policy has to figure out where to take it.

So; we switched to a 3rd policy shape + architecture where it has to output 2 things 1st is which particle to move & second is a number (dt, dx). So, we first used a (multi-discrete) output. i.e. we will have 2 softmax's at the as a last layer of the (π) which is a network (with masking)

But this caused a problem, because we got a gap between the deterministic & stochastic Rollout. (add why)

=> So what we did is a decoding step where we basically stack all the possibilities & choose from them.

ex. (particle 1), (dt1, dx1)
(particle 1), (dt2, dx1)



6 possibilities for each particles.

(0,0): Stays in its place. (1,0): right

(0,+1): forward (-1,0): left

(-1,+1): left up (forward) (+1,+1): Right forward

& then π will choose one number which represents what pt & where should it go.

however, as you might notice 50% of the actions where wasted on $dt=0$ which caused many collision Rejections & thus No training progress so, because choosing what pt to take was part of the policy itself - dt=0 as an option was Removed.
which made the action space 50% smaller- which helped.

~~which that we~~

some notes on the training => we are using PPO - Gymnasium with 15 cores

in the range of $10^{*}dt$ where $dt = \frac{0.03}{9} = \frac{1}{300}$

$= \frac{3.6 \times 10^{-4}}{6} = 1.6 \times 10^{-4}$ sec

~~But~~ more details are yet to come

& we have around 45 particles now, we started with 10, now we are upscaling.

after removing $dt=0$ as an option, we still got stuck in couple of problems. the first one was noticed ~~where~~ & it was because of the constraints which are the following: (discussed in chapter before)

→ weight (2 constraints) → collisions
→ geometry (2 ") → U-boundedness

SO; what happened is we were getting stuck

Now were in 25/07/

I am trying to continue the process of writing what I am working on.

In this writing session I am gonna write down the details of the problem.

→ lets go back to the tuple & start there.

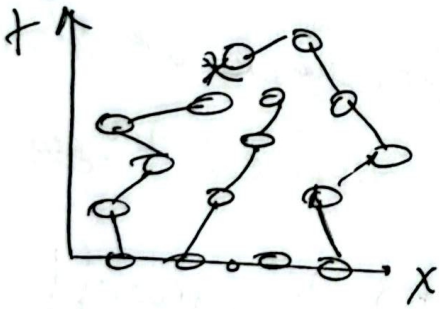
~~Reg~~ $\{S, A, R, \gamma\}$

First. lets try to explain the problem computationally & conceptually.

we are gonna assume we have particles are on top of the ICs. & then these points will keep moving until 2 things. the 1st thing that it will reach the target pt. & then the reward will fire.

But other wise we get a max number of possible iterations & the episode will terminate. (no calculations of u^*)

→ So, lets imagine a scenario where particles start at the ICs & the have to find the treasure & go to get in an optimal way.



Something like this. & this worked fine until we problems with the errors (doc discussed)

so what we decided to do is to have something like circle/rectangle around the * & wherever we get 5 inside the □, we will use them as neighbours to get u^* & no need to have one particle landon it & if they don't get #5 pts inside the Box we used the KNN to get what is probably u^*

To start with the tuple $< R, A, S, \gamma >$

lets start with what not changed. γ, so, for we are still saying that the future is as important as the present. & $\boxed{\gamma = 1}$

which makes the $G_t = r_t + \gamma r_{t+1} + \text{——}$

$$G_t = r_t + r_{t+1} + \text{——}$$

But in our terminal reward case, we will still get one final reward at the termination & $G_t = G_{t+1} =$

$$G_t = r_{term.}$$

which takes me to the second part of the tuple which is the Reward.

the reward so far is terminal and is equal to

$$r_{final} = -\lambda |\hat{u}^* - u^*| - \beta k$$

however, later, I changed this because we started having different parameters for different t^* and it also came from having ~~u~~ dense reward!

So; why did we add a ~~terminal~~ sparse dense reward.

the problem with this starts with having 2 rewards for the initial case where we have the particle to touch the $*$

because what we did is basically say if it touches then r is what we just mentioned.

But if we don't get there we were saying that

$r_{terminal} = -1 - \lambda K$

& that was working fine for small t^*, because the second term was small enough to match this

in other words, we wanted $r_{timeout} > r_{reach}$ ∀ episodes

So; at the $t^*_{small} \simeq 5dt$ for example wherever we touched the t^* -beta*Acc

term was always < -1

because error didn't propagate much yet.

but later $r_{terminate} < r_{bad\ reach}$ ∀ $t^* > ndt$

where n is bigger than the toy problem we talked about

So we had to either make beta smaller which means we're setting for bad accuracy which is something we want, so we had to normalize things & move to box idea we previously discussed

for the normalization

the easy part is of the speed, we'll say $\left(\frac{K}{K_{max}}\right) \cdot w$, & now $\frac{K}{K_{Max}} \leq 1 \ \forall k$

& now w can replace ~~be~~ λ

for beta* accuracy → we can have w_{acc} instead of β & instead of accuracy, we're gonna replace it by $\min\left(\frac{e}{e_{tor}}, 1\right)$ where where we will introduce new parmater that will decide on the accuracy

So, the reward in general for any cases with the box/circle idea is uniform & 1 for all cases

$R = R_{terminal} = -w_{acc}$